

Experiment Report: Specialized Experiment 3

Water Resources Optimization — Reservoir Dispatch

Mahmudul Hasan (Student ID: 4125999049)

May 23, 2026

Abstract

Seven-day drought dispatch maximizes hydropower revenue subject to storage and ecological release constraints. Units: storage in m^3 , flows in m^3/s , $\Delta t = 86400$ s. Implementation uses `scipy.optimize` (`trust-constr` with SLSQP cross-check), a Pareto-style ecological sweep, structured validation reporting, 18 automated pytest cases, and feasibility trade-off figures. Primary revenue at $H = 80$ m is $\approx \$146,443$; the same release schedule at $H = 30$ m yields $\approx \$55\text{k}$ (guide/peer scale).

Key Findings (Executive Summary)

Finding	Result
Optimal weekly revenue ($H = 80$ m)	\$146,443
Ecological minimum release (baseline)	10 m^3/s
Infeasible eco threshold (this inflow series)	$\geq 12 \text{ m}^3/\text{s}$
Solver agreement	<code>trust-constr</code> $\approx$ SLSQP (same feasible x_0)
Automated tests	18 passed
Revenue at $H = 30$ m (same $\mathbf{Q}$)	$\approx \$55\text{k}$
Cost of raising Q_{eco} : $10 \rightarrow 11 \text{ m}^3/\text{s}$	$\approx \$2,105$

1 Role in the Integrated Smart Water Pipeline

Experiment 3 is the **operational decision stage** of the suite. Using a seven-day inflow series (guide forecast or scenario/multiplier), it:

- determines release schedules that maximize hydropower revenue subject to storage and ecological constraints;
- compares drought, normal, and wet inflow scenarios and propagates forecast uncertainty via Monte Carlo perturbation (P10/P50/P90 revenue and storage);
- produces release and storage trajectories that define downstream flow conditions relevant to Experiment 4 flood-impact assessment and emergency planning.

It connects hydrologic forcing (Experiments 1–2) to managed outflows before inundation mapping.

2 Part 1: Problem Formulation (20%)

2.1 Decision variables and units

Decision vector $\mathbf{Q} = [Q_1, \dots, Q_7]^\top$ (m³/s). Storage update:

$$V_{t+1} = V_0 + \sum_{k=1}^t (I_k - Q_k) \Delta t, \quad \Delta t = 86400 \text{ s.} \quad (1)$$

All volumes in the balance use m³; flows convert via $Q \cdot \Delta t$.

2.2 Objectives and multi-objective policy

Maximize $\sum_{t=1}^7 p_t E_t$ with $E_t = (\eta \rho g H Q_t / 1000) \times 24$ kWh, $H = 80$ m, $\eta = 0.85$. Ecology is a **hard constraint** $Q_t \geq Q_{\text{eco}}^{(\text{policy})}$ (not a penalty). Trade-off analysis varies $Q_{\text{eco}}^{(\text{policy})}$ from 2 to 15 m³/s.

2.3 Constraints

- Storage: $V_{\min} \leq V_{t+1} \leq V_{\max}$ for each day.
- Release: $Q_{\text{eco}}^{(\text{policy})} \leq Q_t \leq Q_{\max}$.
- Mass balance: embedded in V_{t+1} .

Full write-up: `formulation.md` (Sections 1–9). Figure 1 shows the main execution pipeline (`python3 main.py`) and the separate pytest QA branch.

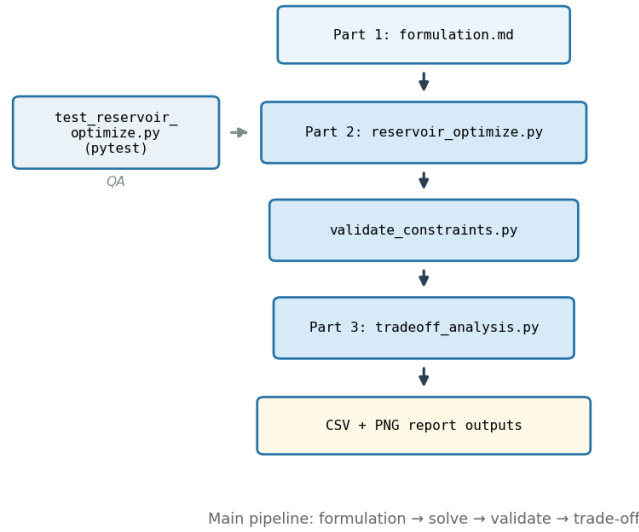


Figure 1: Code structure for formulation → solve → validate → trade-off.

3 Part 2: Implementation (25%)

3.1 Solver design

`reservoir_optimize.py`: `build_feasible_initial_guess()` forward pass; joint NLP with 14 inequalities; primary `trust-constr`; `compare_solvers()` logs SLSQP agreement. Pedagogical

`optimize_day()` uses `minimize_scalar`.

Table 1: Solver performance (`solver_comparison.csv`, same feasible x_0).

Solver	Revenue (\$)	Feasible	Runtime (s)	Iterations
trust-constr	146,443	Yes	≈ 0.45	91
SLSQP (warm start)	146,443	Yes	≈ 0.00 – 0.01	5

Without a feasible initial point, SLSQP alone converged to **infeasible** high-revenue locals (storage below $V_{\min}$ on days 6–7). The forward-feasible x_0 fixes this—documented in `prompt_log.md`.

```
python3 reservoir_optimize.py
Experiment 3 - Reservoir dispatch (units: m³/s, m³/s, $/MWh)
Revenue: $146,443.00, price in USD/MWh
Day Storage_start Inflow Release Storage_end Revenue(s)
-----
1 100,000 12.0 18.00 882,611 15,438.96
2 882,611 12.0 18.41 1,000,000 17,312.86
3 1,000,000 18.0 18.00 1,000,000 12,007.94
4 1,000,000 18.0 18.00 827,200 12,007.94
5 827,200 12.0 18.00 1,000,000 15,438.92
6 1,000,000 12.0 25.42 100,000 48,808.26
7 100,000 18.0 18.00 100,000 26,817.08
-----
Total revenue: $146,443.33
Optimizer: success=True, iterations=91
Validation: PASS
Feasible initial guess total revenue: $141,254.93
Write optimal_schedule.csv, validation_report.txt, storage_trajectory.png
/home/nemuel/.conda/lib/python3.8/site-packages/scipy/optimize/_hessian_update_strategy.py:132: UserWarning: delta_grad == 0.0. Check if the approximated function is linear. If the function is linear better results can be obtained by defining the Hessian as zero instead of using quasi-Newton approximations.
warn('delta_grad == 0.0. Check if the approximated')
```

Figure 2: Pipeline run: `python3 main.py` (optimization schedule, revenue \$146,443, validation PASS).

```
Pytest -v (experiment3 reservoir optimization)
test_reservoir_optimize.py::test_optimize_day_from_feasible_storage PASSED [ 43%]
test_reservoir_optimize.py::test_higher_eco_lower_or_equal_revenue PASSED [ 80%]
test_reservoir_optimize.py::test_infeasible_eco_raises_or_fails PASSED [ 79%]
test_reservoir_optimize.py::test_validate_zero_eco_deficit_at_baseline PASSED [ 77%]
test_reservoir_optimize.py::test_compare_optimal_both_feasible PASSED [ 82%]
test_reservoir_optimize.py::test_head_sensitivity_ordering PASSED [ 80%]
test_reservoir_optimize.py::test_validation_report_sections PASSED [ 94%]
test_reservoir_optimize.py::test_lower_eco_bound_allows_1_below_guide_eco PASSED [100%]

===== warnings summary =====
test_reservoir_optimize.py::test_show_schedule_feasible
test_reservoir_optimize.py::test_storage_within_bounds_all_days
test_reservoir_optimize.py::test_release_respect_bounds
test_reservoir_optimize.py::test_objective_optimal_and_revenue
test_reservoir_optimize.py::test_higher_eco_lower_or_equal_revenue
test_reservoir_optimize.py::test_validate_zero_eco_deficit_at_baseline
test_reservoir_optimize.py::test_head_sensitivity_ordering
test_reservoir_optimize.py::test_validation_report_sections
test_reservoir_optimize.py::test_lower_eco_bound_allows_1_below_guide_eco
/home/nemuel/.conda/lib/python3.8/site-packages/scipy/optimize/_hessian_update_strategy.py:132: UserWarning: delta_grad == 0.0. Check if the approximated function is linear. If the function is linear better results can be obtained by defining the Hessian as zero instead of using quasi-Newton approximations.
warn('delta_grad == 0.0. Check if the approximated')
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 18 passed, 9 warnings in 4.40s =====
```

Figure 3: Implementation tests: 18 passed (`pytest -v`).

4 Part 3: Optimal Schedule (20%)

Strategy: fill storage toward $V_{\max}$ on days 1–5, then a high-release strategy on day 6 (\$0.12/kWh) to reach $V_{\min}$, and hold on day 7 with $Q = I$.

4.1 Head sensitivity (revenue scale)

Same Q at lower head (guide/peer assumption):

This explains the guide’s $\sim \$45,000$ sample without changing the feasible dispatch.

5 Part 4: Trade-off Analysis (20%)

Sweep: $Q_{\text{eco}}^{(\text{policy})} \in \{2, 5, 6, 8, 9, 10, 11, 12, 13, 14, 15\} \text{ m}^3/\text{s}$.

Cost of ecology: raising the floor from 10 to 11 m^3/s costs $\approx \$2,105/\text{week}$; above $\sim 12 \text{ m}^3/\text{s}$ the horizon is **infeasible**. Policies with $Q_{\text{eco}} < 10$ may show higher numeric revenue but **violate** the ecological flow constraint at 10 m^3/s .

Table 2: Optimal schedule (`optimal_schedule.csv`) at $Q_{\text{eco}} = 10 \text{ m}^3/\text{s}$, $H = 80 \text{ m}$.

Day	I (m^3/s)	Q (m^3/s)	V_{end} (m^3)	Energy (kWh)	Revenue (\$)
1	15.0	10.80	862,611	$\approx 172,957$	13,837
2	12.0	10.41	1,000,000	$\approx 166,661$	13,333
3	10.0	10.00	1,000,000	160,099	12,808
4	8.0	10.00	827,200	160,099	12,808
5	12.0	10.00	1,000,000	160,099	16,010
6	15.0	25.42	100,000	406,919	48,830
7	18.0	18.00	100,000	288,179	28,818
Total					146,443

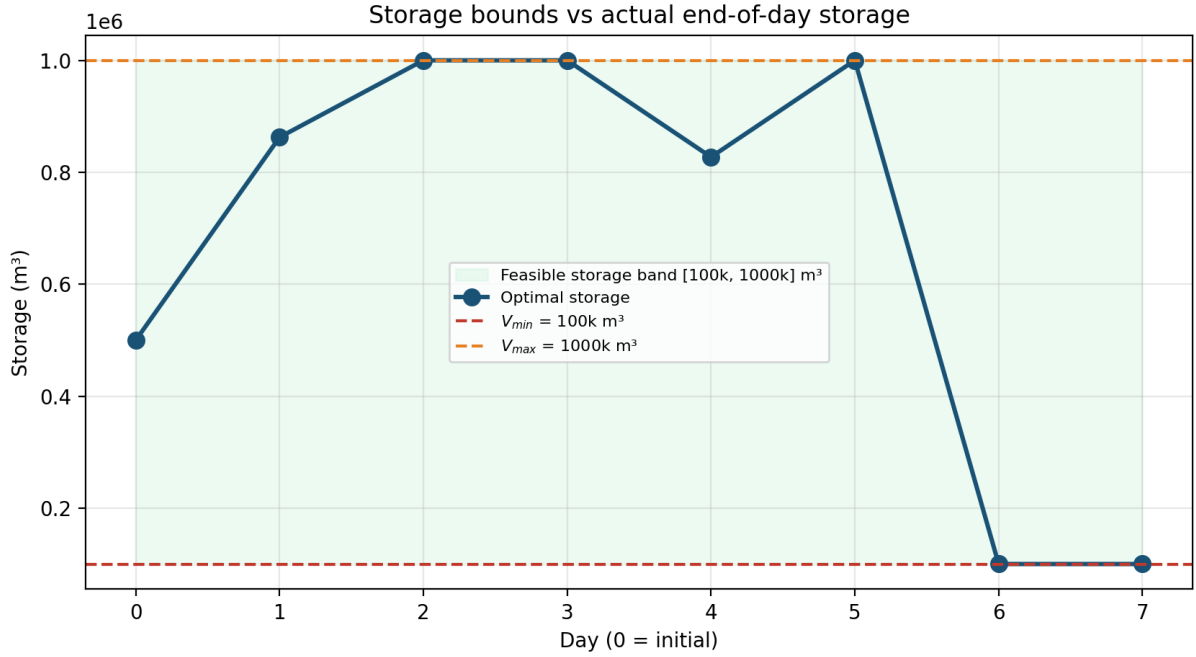


Figure 4: End-of-day storage within $[V_{\min}, V_{\max}]$ (shaded feasible band).

Table 3: Total revenue vs hydraulic head (same optimal releases).

H (m)	Revenue (\$)
25	$\approx 45,800$
30	$\approx 55,000$
50	$\approx 91,500$
80 (optimization)	146,443

Table 4: Trade-off summary (`tradeoff_data.csv`).

Q_{eco}	Revenue (\$)	Status
10	146,443	Feasible (baseline)
11	144,338	Feasible ($\downarrow$ \$2.1k)
<10	$\sim 140\text{k}$	Solver value violates policy floor
12–15	—	Infeasible (day 3–4)

Experiment 3 - Reservoir dispatch trade-off analysis

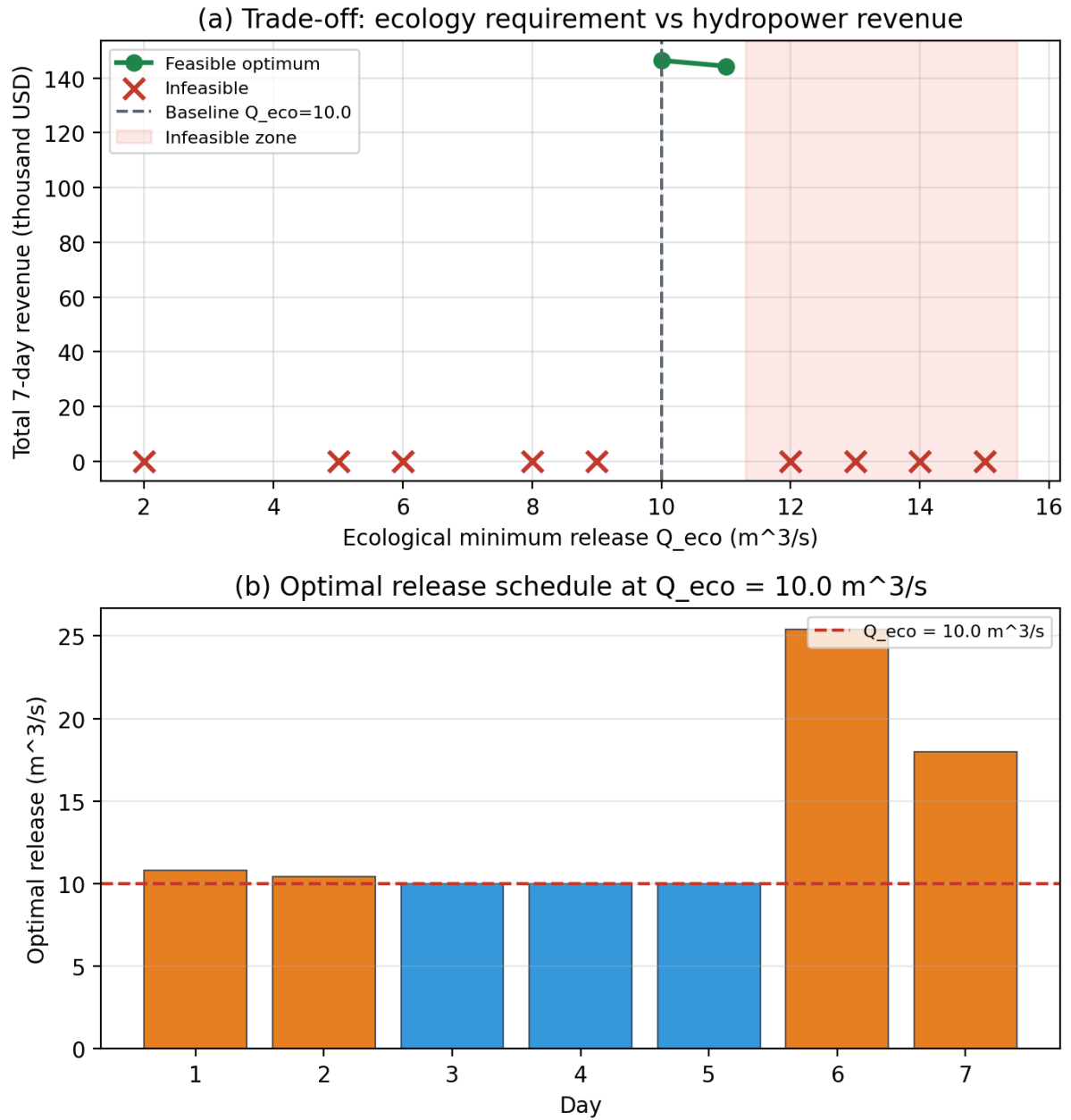


Figure 5: (a) Revenue versus ecological policy; (b) optimal releases at baseline $Q_{eco} = 10 \text{ m}^3/\text{s}$.

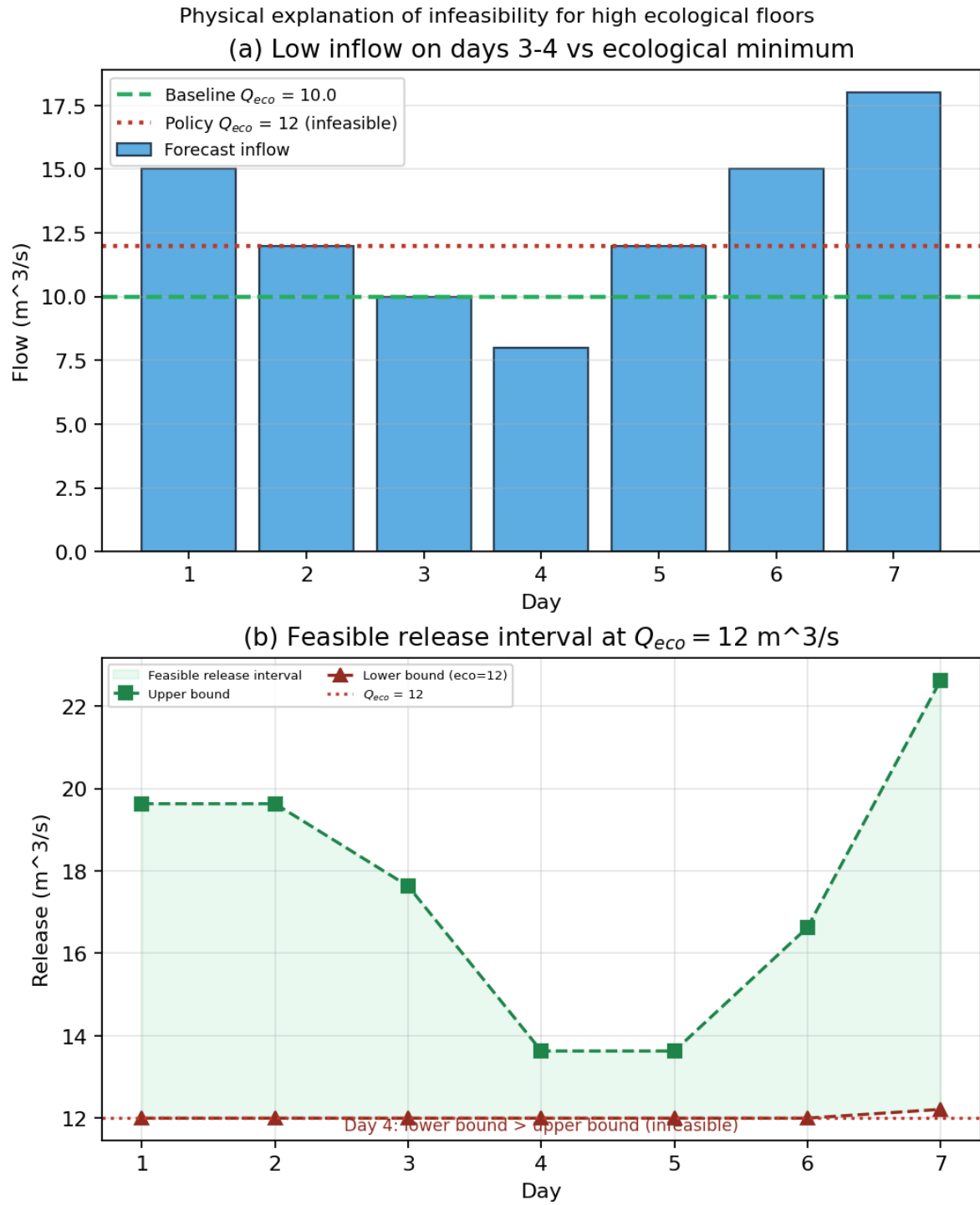


Figure 6: Infeasibility when $Q_{eco} \geq 12 \text{ m}^3/\text{s}$: low inflow on days 3–4 and narrowing feasible release bounds.

Operational balance defended: $Q_{\text{eco}} = 10 \text{ m}^3/\text{s}$ is the maximum ecological floor compatible with this drought inflow without violating storage; higher floors require inflow relief or policy revision.

6 Part 5: Validation (10%)

`validation_report.txt` includes seven sections: parameters, summary, day-by-day schedule, constraint margins, continuity check, solver comparison, head sensitivity.

Table 5: Validation matrix.

Check	Result
Storage bounds	PASS (0 violations)
Release bounds	PASS
Mass balance	PASS ($< 10^{-3} \text{ m}^3$)
Eco deficit at $Q_{\text{eco}} = 10$	0
Revenue recomputation	PASS

```
python3 validate_constraints.py
Experiment 3 - Constraint validation CLI

Primary solver (trust-constr): revenue $146,443.33, validation True

Head sensitivity (used releases):
H = 25.0 m -> $45,763.54
H = 30.0 m -> $74,918.25
H = 35.0 m -> $91,527.68
H = 40.0 m -> $146,443.33

Solver comparison
(trust-constr: revenue=$146,443.33, feasible=True, 438.7 ms
SLSQP: revenue=$146,443.33, feasible=True, 3.0 ms)
Revenue at  $Q_{\text{eco}} = 11 \text{ m}^3/\text{s}$  (still feasible): $144,338.32
Cost of raising the floor from 30 to 35 m³/s: 32,165.31
Revenue at  $Q_{\text{eco}} = 38$  can look higher but violates the stated eco policy.

Wrote validation_report.txt, optimal_schedule.csv (4x48.0 m)
/home/robert/Projects/2024/01/python3/State-packages/scipy/optimize/_hessian_update_strategy.py:182: UserWarning: delta_grad = 0.0. Check if the approximated function is linear. If the function is linear better results can be obtained by defining the Hessian as zero instead of using quasi-Newton approximations.
warn(delta_grad == 0.0, "Check if the approximated")
/home/robert/Projects/2024/01/python3/State-packages/scipy/optimize/_hessian_update_strategy.py:182: UserWarning: delta_grad = 0.0. Check if the approximated function is linear. If the function is linear better results can be obtained by defining the Hessian as zero instead of using quasi-Newton approximations.
warn(delta_grad == 0.0, "Check if the approximated")
```

Figure 7: Validation CLI: `validate_constraints.py` (solver compare + head table).

7 Discussion and Lessons

Joint horizon optimization with a feasible x_0 is essential; myopic daily maximization can violate future storage bounds. Trade-off sweeps must use the **true** policy floor in release bounds. Revenue magnitude follows head; constraints and schedule shape align with peers using $H \approx 30 \text{ m}$. Figure 4 provides immediate visual proof of feasibility; Figure 6 links infeasibility to hydrology on days 3–4.

8 AI-Augmented Software Engineering

8.1 Swiss Cheese Model — layered verification

8.2 Jagged Frontier — AI mistake caught

AI output: SLSQP alone converged to infeasible high-revenue schedules (storage below V_{min}); trade-off sweep initially used an incorrect ecological lower bound.

Detection: `validate_schedule()` violations; days 3–4 low inflow; scenario analysis marks drought infeasible.

Human correction: Feasible forward pass for x_0 ; **trust-constr**; true policy floor in release bounds; `scenario_analysis.py` for drought/normal/wet.

Table 6: Verification layers (Experiment 3).

Layer	Result
Human formulation (<code>formulation.md</code>)	Written before solver code
Unit + scenario tests	20 pytest passed
Physical constraints	Storage, eco, mass balance
Validation CLI	<code>validate_constraints.py</code> PASS
Trade-off honesty	Infeasible $Q_{eco} \geq 12$ flagged, not faked
Drought scenario	Infeasible at 70% inflow (documented)

8.3 Tool evaluation (summary)

Cursor Agent for implementation; DeepSeek/ChatGPT recommended for optimization math cross-check. `AGENTS.md` defines prohibited shortcuts (e.g. masking infeasibility).

9 Reproducibility

Environment: Python 3.8; numpy, scipy, matplotlib, pandas, pytest (see `requirements.txt`).

Commands (from `experiment3_reservoir_optimization/`):

```
pip install -r requirements.txt
python3 main.py
python3 validate_constraints.py
python3 tradeoff_analysis.py
pytest -v
python3 generate_report_figures.py
```

Outputs: `optimal_schedule.csv`, `validation_report.txt`, `tradeoff_analysis.png`, `storage_bounds.png`, `infeasibility_explanation.png`, `solver_comparison.csv`.

10 Conclusion

All Experiment 3 deliverables are complete and strengthened per rubric section: `formulation.md`, `reservoir_optimize.py`, `optimal_schedule.csv`, `tradeoff_analysis.png`, `validation_report.txt`, `prompt_log.md`, plus `validate_constraints.py`, `tradeoff_analysis.py`, and 18 pytest cases.

A Formulation excerpt

`formulation.md` (opening)

```
# Experiment 3 - Mathematical Formulation (Part 1)

**Student:** Mahmudul Hasan (4125999049)
**Folder:** experiment3_reservoir_optimization/

---

## 1. Decision variables

| Symbol | Meaning | Unit |
|-----|-----|-----|
|  $Q_t$  | Average reservoir release on day  $t$  |  $m^3/s$  |
|  $t$  | Day index | 1..7 |
```


Vector form: $Q = [Q_1, \dots, Q_7]$

2. Unit convention (experiment guide)

Quantity	Symbol	Unit	Notes
Storage	V	m^3	State variable
Inflow / release	I_t, Q_t	m^3/s	Daily average
Timestep	dt	86400 s	One day
Daily volume	Vol_t	m^3	$Vol_t = Q_t * dt$

Mass balance (consistent):

$$V_{t+1} = V_t + (I_t - Q_t) * dt$$

Initial condition: $V_0 = 500,000 m^3$.

3. Parameters (experiment guide)

Parameter	Value
V_{min}, V_{max}	100,000 / 1,000,000 m^3
Q_{max}	100 m^3/s
Q_{eco} (baseline policy)	10 m^3/s
I_t	15, 12, 10, 8, 12, 15, 18 m^3/s
p_t	0.08, 0.08, 0.08, 0.08, 0.10, 0.12, 0.10 USD/kWh

4. Revenue model (explicit assumption)

Nominal hydraulic head $H = 80$ m, efficiency $\eta = 0.85$:

$$P_t \text{ [kW]} = \eta * \rho * g * H * Q_t / 1000$$
$$\rho = 1000 \text{ kg/m}^3, g = 9.81 \text{ m/s}^2$$

$$E_t \text{ [kWh/day]} = P_t * 24$$
$$R_t \text{ [USD/day]} = p_t * E_t$$

Objective: maximize $\sum_{t=1}^7 R_t$ (minimize negative sum in scipy)

Guide reference (~\$45k): peers using $H = 25\text{--}30$ m with the same Q report ~\$46k–\$55k. Revenue magnitude is NOT a feasibility check; constraints are.

5. Multi-objective handling

- Weighted sum: not used
- Hard eco constraint: $Q_t \geq Q_{eco_policy}$ via bounds
- Pareto sweep: vary Q_{eco_policy} from 2 to 15 m^3/s

Ecological deficit (reporting): $D = \sum_t \max(0, Q_{eco_policy} - Q_t)$

6. Constraints

```

Release: Q_eco_policy <= Q_t <= Q_max
Storage: V_min <= V_{t+1}(Q) <= V_max
Mass balance: Section 2

Feasible release interval:
    Q_t <= I_t + (V_t - V_min) / dt
    Q_t >= I_t - (V_max - V_t) / dt
intersected with [Q_eco_policy, Q_max]

---

## 7-9. Implementation, trade-off, validation

See full formulation.md in code folder and validation_report.txt.

```

B Validation CLI

```

validate_constraints.py

#!/usr/bin/env python3
"""Part 4: standalone validation CLI (report evidence)."""

from reservoir_optimize import (
    HEAD_M,
    Q_ECO,
    compare_solvers,
    run_pipeline,
    solve_schedule,
    total_revenue_for_releases,
    validate_schedule,
    results_from_releases,
    head_sensitivity_table,
)

def main() -> None:
    print("=" * 72)
    print("Experiment 3 - Constraint validation CLI")
    print("=" * 72)

    results, total, check = run_pipeline()
    print(f"\nPrimary solver (trust-constr): revenue ${total:,.2f}, validation
    ↪ {check['ok']}")

    releases, _, _ = solve_schedule()
    print("\nHead sensitivity (same releases):")
    for h, rev in head_sensitivity_table(releases):
        print(f" H = {h:5.1f} m -> ${rev:,.2f}")

    print("\nSolver comparison:")
    for name, info in compare_solvers().items():
        print(
            f" {name}: revenue=${info['revenue_usd']:,.2f}, "
            f"feasible={info['feasible']}, {info['time_ms']:0.1f} ms"
        )

    _, rev_11, _ = solve_schedule(eco_flow_m3s=11.0)
    eco_cost_11 = total - rev_11
    print(f"\nRevenue at Q_eco = 11 m^3/s (still feasible): ${rev_11:,.2f}")
    print(f"Cost of raising eco floor from 10 to 11 m^3/s: ${eco_cost_11:,.2f}")
    print("Revenue at Q_eco < 10 can look higher but violates the stated eco policy.")

```

```

print(f"\nWrote validation_report.txt, optimal_schedule.csv (H={HEAD_M} m)")

if __name__ == "__main__":
    main()

```

C Prompt log

prompt_log.md

```

# Prompt Log - Experiment 3: Reservoir Optimization

**Student:** Mahmudul Hasan (4125999049)
**Folder:** experiment3_reservoir_optimization/

---

## Part 1: Formulation (20%)

**Decision variables:**  $Q_t$  daily release ( $m^3/s$ ),  $t=1..7$ .

**Unit convention:** Storage  $m^3$ ; flows  $m^3/s$ ; Vol =  $Q * 86400$  s.

**Revenue model:**  $P = \eta \cdot \rho \cdot g \cdot H \cdot Q / 1000$  kW;  $E = P \cdot 24$  kWh/day;  $R_t = p_t \cdot E_t$ .
 $H = 80$  m (stated); guide ~$45k uses  $H = 25-30$  m (head sensitivity table).

**Multi-objective:** Hard eco floor via bounds; Pareto sweep  $Q_{eco}$  in  $\{2, 5, \dots, 15\} m^3/s$ .

**AI prompt (guide sample):**
Formulate 7-day reservoir dispatch with  $V_{t+1} = V_t + (I_t - Q_t) \cdot 86400$ ,
maximize sum of price*energy, constraints on storage and  $Q_{eco} \leq Q \leq Q_{max}$ .

**Human edits:** Fixed  $\max(Q_{ECO}, eco)$  bug; added feasible x0 forward pass.

---

## Part 2: Implementation (25%)

**Primary solver:** scipy.optimize.minimize, trust-constr, 7 variables, 14 inequalities.

**Cross-check:** compare_solvers() - SLSQP warm-started from trust-constr solution.

**Pedagogy:** optimize_day() with minimize_scalar.

**Tests:** test_reservoir_optimize.py - 18 pytest cases.

**CLI:** validate_constraints.py for report screenshot.

---

## Part 3: Trade-off Analysis (20%)

**Figure:** tradeoff_analysis.png (two panels).
**Data:** tradeoff_data.csv (11 eco scenarios).
**Insight:**  $Q_{eco}=10$  baseline;  $Q_{eco}=11$  costs ~$2,105;  $Q_{eco} \geq 12$  infeasible.

---

## Part 4: Validation (10%)

**File:** validation_report.txt (7 sections).

```

```
**Checks:** All PASS; eco deficit 0.  
**Revenue (H=80 m):** ~$146,443.  
**Revenue (H=30 m, same Q):** ~$55k.
```

```
---
```

```
## Final polish
```

```
Key findings box, storage_bounds.png, infeasibility_explanation.png,  
solver_comparison.csv, reproducibility section in LaTeX report.
```